



JavaScript Asynchronous Programming

Callbacks, Promises, and Async/Await

● callbacks • ● promises • ● async/await



Mastering JavaScript Asynchronous Patterns

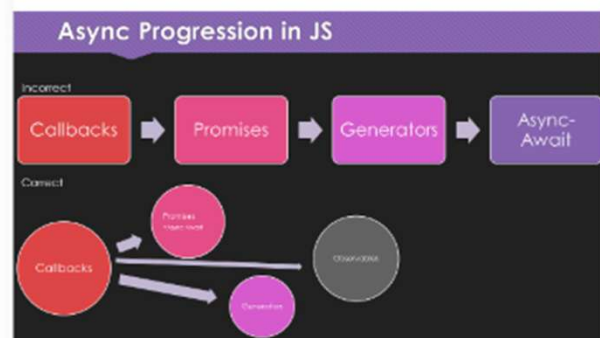
Understanding Asynchronous Programming

JavaScript is **Single-Threaded**

Executes one operation at a time. Asynchronous patterns allow handling long-running operations without blocking the main thread.

Async Patterns Handle:

-  API calls
-  File reading
-  Timers
-  Event handling



Evolution

🔗 What is a Callback?

✓ Definition

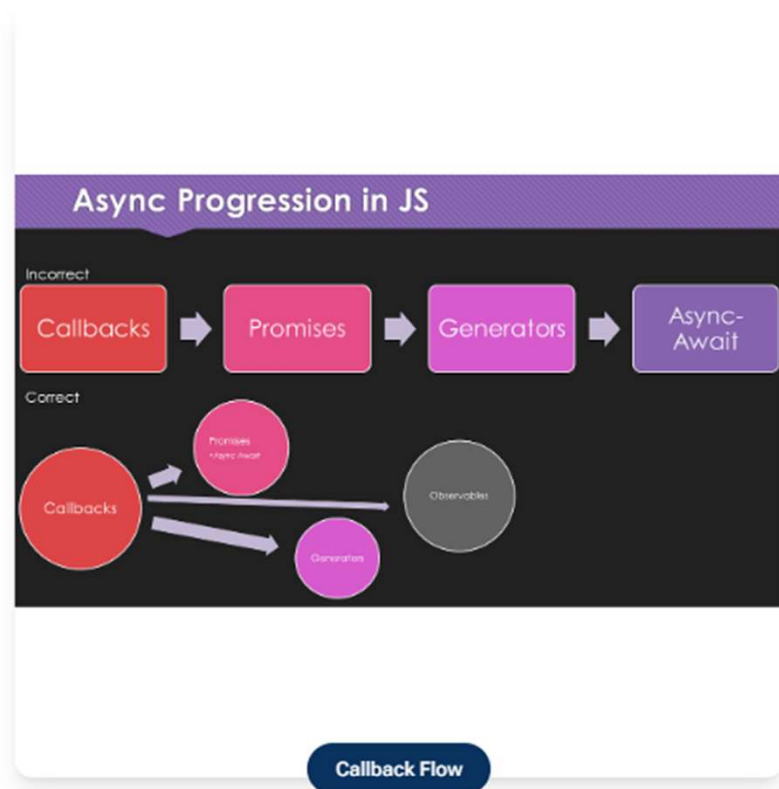
A callback is a function **passed as an argument** to another function and executed later.



👉 In simple words:

A callback is a function that runs after another function finishes.

<> Function → executes callback



🔊 Why Callbacks are Needed

🧠 JavaScript Nature

JavaScript is **asynchronous** and **single-threaded**

📋 Callbacks Handle:

☁️ API calls

📄 File reading

🕒 Timers

👤 Event handling

<> ✅ Basic Syntax

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}
```

➔ Function receives callback as parameter

🔊 Why Callbacks are Needed

🧠 JavaScript Nature

JavaScript is **asynchronous** and **single-threaded**

📋 Callbacks Handle:

☁️ API calls

📄 File reading

🕒 Timers

👤 Event handling

<> ✅ Basic Syntax

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}
```

➔ Function receives callback as parameter

▶ Callback Example

<> ✓ Example Code

```
function processUserInput(name, callback) {  
  console.log("Processing user: " + name);  
  callback();  
}  
  
function showMessage() {  
  console.log("Process completed!");  
}  
  
processUserInput("Vivek", showMessage);
```

📄 ✓ Output

```
Processing user: Vivek  
Process completed!
```

↗ Function calls

Passes callback as
argument

🕒 Execution order

Main function →
Callback

⚠️ 🚨 Callback Hell

❌ Example (Callback Hell)

```
setTimeout(() => {  
  console.log("Step 1");  
  setTimeout(() => {  
    console.log("Step 2");  
    setTimeout(() => {  
      console.log("Step 3");  
    }, 1000);  
  }, 1000);  
, 1000);
```



Problem 1

Hard to read



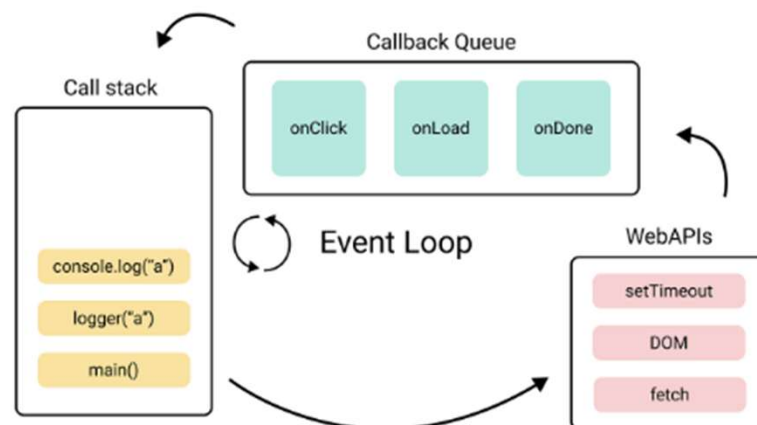
Problem 2

Hard to debug



Problem 3

Hard to
maintain



👉 **Solution: Promises**

Cleaner, more maintainable async code

⚙️ What is a Promise?

✓ Definition

A Promise is an **object** that represents the **future result** of an asynchronous operation.

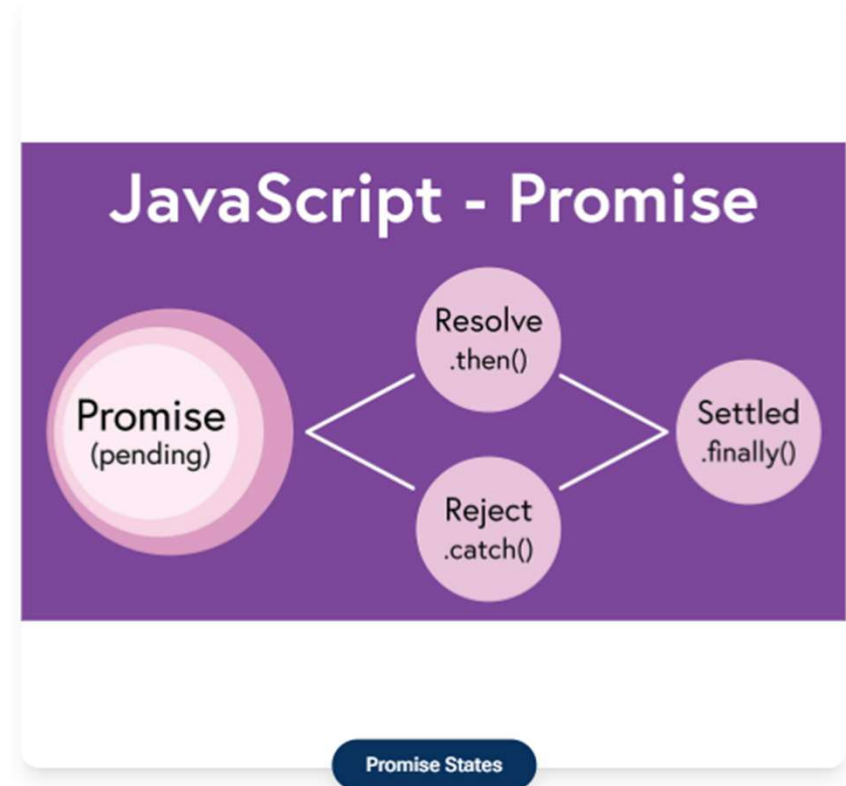
💡 Simple meaning:

A Promise is a **guarantee** that something will happen later.



Future value

Represents eventual completion or failure



🕒 Promise States

💡 A promise has 3 states:



Pending

initial state



Fulfilled

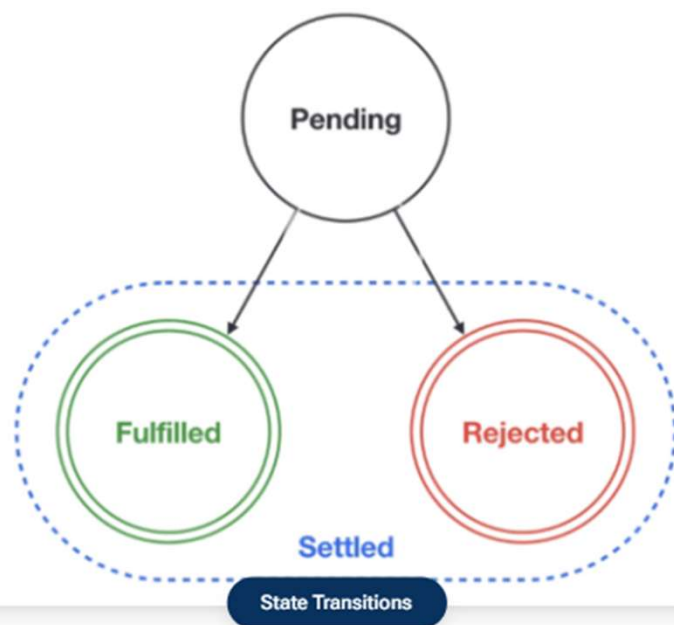
operation successful



Rejected

operation failed

Promise States



🕒 Creating & Consuming a Promise

✂️ ✅ Creating a Promise

```
let myPromise = new Promise((resolve, reject) => {  
  // Async operation  
  let success = true;  
  
  if (success) {  
    resolve("Task completed");  
  } else {  
    reject("Task failed");  
  }  
});
```

✅ `new Promise()` creates the object

▶️ ✅ Consuming a Promise

```
myPromise  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

➔ `.then()`

Handles success

⚠️ `.catch()`

Handles failure

🔗 Chained method calls

➡ Promise Chaining

🕒 ✅ Real Example (Timer)

```
function waitTwoSeconds() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Done after 2 seconds");  
    }, 2000);  
  });  
}  
  
waitTwoSeconds().then(msg => console.log(msg));
```

🕒 Simulates async operation

🔧 ⚡ Promise Chaining

```
fetchData()  
  .then(data => processData(data))  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

✅ ✅ Benefits

Avoids callback hell

👤 ✅ Clean Code

Flatter structure

⚡ What is Async/Await?

✓ Definition

Async/Await is **syntactic sugar** over Promises that makes asynchronous code look **synchronous**.

💡 Simple meaning:

Async/Await makes Promises **easier to write and read**.



Modern JavaScript
Cleaner asynchronous patterns

📋 Rules

- 1 async → before function**
Mark function as asynchronous
- 2 await → inside async function only**
Cannot use await in regular functions
- 3 await waits for promise**
Pauses execution until promise resolves

? What is Async-Await?



Modern JavaScript Feature

Makes working with asynchronous code **easier** and **more readable**



Built on Promises

It is built **on top of Promises** and helps write asynchronous code in a **synchronous style**



Syntactic Sugar

Async/Await is **syntactic sugar** over Promises that makes code look **synchronous**

1

async

Makes function return a Promise

2

await

Pauses execution until Promise settles

1 `async` Keyword

✔️ Definition

An `async` function **always returns a Promise**.

💡 Key Point

Even if you return a **normal value**, JavaScript **automatically wraps it** inside a Promise.

➔ No explicit Promise creation needed

<> ✨ Syntax

```
async function myFunc() {  
  return "Hello";  
}
```

✨ What it actually returns

```
Promise { "Hello" }
```

✨ JavaScript wraps "Hello" in Promise automatically

1 **async Keyword – Important Points**



Written Before Function

async is written **before** a function keyword

```
async function myFunc() {}
```



Returns Promise Automatically

Makes the function **return a Promise** automatically

✓ No explicit Promise needed



Allows await Inside

Enables the use of **await** inside the async function

```
await somePromise;
```



Cannot Use await Without async

Cannot use await outside an async function

⌚ Syntax Error if used alone

2 await Keyword

✓ Definition

await **pauses the execution** of its surrounding async function until the Promise is **settled** (resolved or rejected).

🔍 Meaning of "settled"

✓ **Fulfilled (resolved)**
Promise completed successfully

✗ **Rejected**
Promise failed

<> ♦ Syntax

```
async function myFunc() {  
  let result = await somePromise;  
}
```

⏸ Pauses Execution

The function waits at the **await** line until the Promise settles



Only inside async



Non-blocking

2 **await** Keyword

✓ Definition

await **pauses the execution** of its surrounding **async** function until the Promise is **settled** (resolved or rejected).

🔍 Meaning of "settled"

✓ **Fulfilled (resolved)**
Promise completed successfully

✗ **Rejected**
Promise failed

<> ♦ Syntax

```
async function myFunc() {  
  let result = await somePromise;  
}
```

⏸ Pauses Execution

The function waits at the **await** line until the Promise settles



Only inside **async**



Non-blocking

▶ Example with Explanation

<> Complete Example

```
function getData() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Data received");  
    }, 2000);  
  });  
}  
  
async function fetchData() {  
  console.log("Fetching...");  
  let result = await getData();  
  console.log(result);  
}  
  
fetchData();
```

📈 Execution Flow

- 1 "Fetching..." prints
Immediate execution
- 2 await getData() pauses
Function waits for Promise
- 3 After 2 seconds
Promise resolves
- 4 "Data received" prints
Execution resumes

▶ Example with Explanation

<> Complete Example

```
function getData() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve("Data received");  
    }, 2000);  
  });  
}  
  
async function fetchData() {  
  console.log("Fetching...");  
  let result = await getData();  
  console.log(result);  
}  
  
fetchData();
```

📈 Execution Flow

- 1 "Fetching..." prints
Immediate execution
- 2 await getData() pauses
Function waits for Promise
- 3 After 2 seconds
Promise resolves
- 4 "Data received" prints
Execution resumes

→ Why Use Async-Await?

❌ Without Async/Await

🔗 **.then().then().then()**

Chaining method calls

📦 **Callback Hell**

Deep nesting problems

🚫 **Hard to read**

Complex structure

🐛 **Hard error handling**

Requires .catch()

✅ With Async/Await

<> **Clean sequential code**

Reads like synchronous

↕ **No nesting**

Flat structure

👁 **Easy to understand**

Intuitive flow

🛡 **Use try-catch**

Familiar error handling



Error Handling with Async/Await

✔ Use try...catch

```
async function fetchData() {  
  try {  
    let result = await somePromise;  
    console.log(result);  
  } catch (error) {  
    console.log(error);  
  }  
}
```

✔ try block

Attempt operation

❗ catch block

Handle errors



How it works

1

Execute code in try

Attempt the async operation

2

Catch errors

Handle any thrown exceptions

3

Synchronous-like

Looks like traditional try-catch